周辺技術から理解する

Swift SDK for Android 入門

Android アプリ上で Swift が動くまでの仕組みを、周辺技術から読み解く

yamaken / 山口賢登 / @yamakentoc

Android アプリから Swift が呼ばれるとき、何が起きているのか？

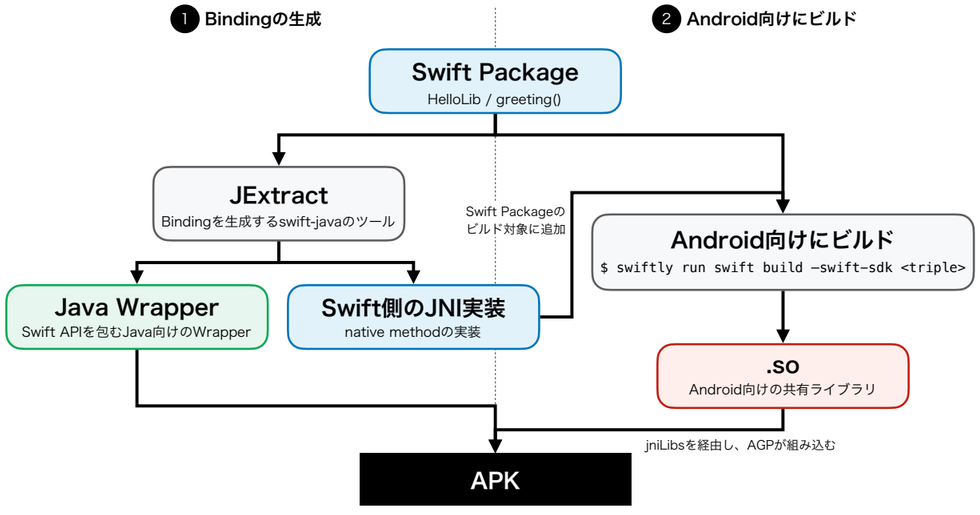
Swift SDK for Android を用いることで、Android アプリ上で Swift のコードを呼び出すことが可能になりました。公式のサンプルを動かすだけであれば簡単ですが、どのような流れで Swift の処理が呼ばれるのかを調べていくと、馴染みの薄い概念やツールが多く登場します。本記事では、Swift SDK for Android に登場する概念やツールを整理しつつ、どのような仕組みで Swift のコードが Android アプリケーション上で動くのかを解説します。

ビルド時の流れを見る

まずは、ビルド時にどのような工程があり、図に登場する技術がそれぞれどこに関わるのかを全体像から確認します。次ページ以降では、各技術の役割を順に掘り下げます。読み進めたあとにもう一度この図へ戻ると、技術同士のつながりも捉えやすくなります。

Swift で書いた処理を Android アプリへ組み込むまでには、大きく分けて「Binding の生成」と「Android 向けのビルド」という 2 つの工程があります。ここでは、次の `greeting()` 関数を公開する Swift Package の HelloLib を例に見ていきます。

```swift
public func greeting() -> String {
    "Hello from Swift!"
}
```



1. Binding の生成

JExtract は Swift Package の公開 API を読み取り、Kotlin / Java から呼び出すための Binding を生成します。生成されるのは、Android 側で利用する Java Wrapper と、Java 側の呼び出しを元の Swift 関数へ中継する Swift 側の JNI 実装です。

2. Android 向けにビルド

生成された Swift 側の JNI 実装を Swift Package のビルド対象に追加し、Swift SDK for Android を用いて Android 向けにビルドします。これにより、元の Swift コードと Swift 側の JNI 実装を含む共有ライブラリである .so が生成され、jniLibs を経由して APK に組み込まれます。

Swift SDK for Android を試すには？

本記事では周辺技術や呼び出しの仕組みに焦点を当て、環境構築などの具体的な手順は省略しています。
Swift SDK for Android を試してみたい方は、筆者の Qiita 記事「Swift SDK for Android を完全に理解する」をご覧ください。
環境構築やビルド、サンプルアプリの詳細な解説なども載せています。

Android 向けのビルドを可能にする

Swift SDK for Android

Swift のコードを Android アプリから利用するには、まず Swift のコードを Android 向けにビルドできなければなりません。その役割を担うのが Swift SDK for Android です。
Swift SDK for Android は、Swift toolchain で Android 向けのコンパイルを可能にするための拡張用 SDK です。Android 向けに必要なライブラリ、ヘッダー、設定ファイルなどを提供し、Swift Package を Android のターゲット向けにビルドできるようにします。

```
$ swiftly run swift build \
--swift-sdk aarch64-unknown-linux-android28
```

実際のビルドは、通常の Swift Package のビルドと同じように swift build で行います。Android 向けにビルドする場合は、--swift-sdk で Android 向けのターゲットを指定します。ここで指定する Android の識別子は Target Triple と呼ばれます。

重要なのは、Swift SDK for Android の役割は「Swift を Android 向けにビルドできるようにすること」だという点です。Kotlin / Java から Swift を呼び出すための橋渡しは、後述する JNI や JExtract が担います。Swift SDK for Android が発表された当初は、Swift SDK for Android だけを利用することで Swift のロジックが Android から呼び出せるという点ばかり目立っていたように見えますが、それを実現するには Swift SDK for Android 単体ではできず、さまざまな技術を利用した結果可能になります。あくまで Swift を Android 向けにビルドできるようにする拡張でしかないのです。

Swift toolchain を管理する

swiftly

Swift SDK for Android は特定バージョンの Swift toolchain に合わせて作られています。そのため、それを利用するには通常の Xcode に付属する toolchain ではなく、Swift SDK for Android と同じバージョンの toolchain を用意する必要があります。
swiftly は toolchain のインストールやバージョン切り替えを行うための管理ツールで、Swift SDK for Android を利用するにはこれの利用が推奨されています。

```
# 特定のバージョンをインストール
$ swiftly install 6.3.3

# 指定したバージョンを使用
$ swiftly use 6.3.3
```

swiftly use でバージョンを選択すると、通常の swift build でもその toolchain が利用されます。swiftly run swift build を使うと、選択中の toolchain で実行することをコマンド上でも明示できます。本記事では、利用する toolchain を明確にするため swiftly run を使用しています。

Android 向けのビルド環境を提供する

Android NDK

Android NDK は、Android アプリにネイティブコードを組み込むための開発キットです。Android で C/C++ などのネイティブコードを扱うためによく使われます。
Swift SDK for Android は、Swift コードを Android 向けにクロスコンパイルする際に、NDK が提供するヘッダーやツールを利用します。そのため NDK に依存しており、Swift SDK for Android を利用するには NDK もインストールする必要があるのです。

Android アプリに組み込まれる共有ライブラリ

.so

.so（Shared Object）は、Android/Linux 系で使われる共有ライブラリのファイル形式です。Android アプリへ Swift の処理を組み込む際、Swift コードは Android 向けの共有ライブラリとしてビルドされます。この中に Swift で書いた処理や、Kotlin / Java から呼び出すための橋渡しとなる Swift 側の JNI 実装が含まれます。
`greeting()` を含んだ HelloLib パッケージから生成された .so の中身を大まかに整理すると、次のようになります。

```
libHelloLib.so
 ├── Swift Package で定義した API
 │    └── greeting()
 │
 ├── JExtract が生成した Swift 側の JNI 実装
 │    └── 公開されたシンボル (Java_com_example_...)
 │
 └── 実行に必要なメタデータやビルド情報など
```

Swift Package から .so 単体を生成してみる

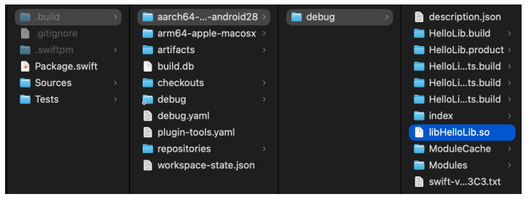
.so は Android アプリ全体をビルドしなくても、Swift Package から単体で生成できます。ここでは、最小構成の Swift Package を作成して確認してみます。

```
$ mkdir HelloLib
$ cd HelloLib
$ swiftly run swift package init --type library
```

続いて、Package.swift のライブラリを dynamic library として定義します。

```swift
products: [
    .library(
        name: "HelloLib",
        type: .dynamic, // 指定
        targets: ["HelloLib"]
    )
]
```

この状態で前述したように Android 向けに swift build を行うと、成果物として .so が生成されていることがわかります。



この結果からわかることは、macOS 向けに動的ライブラリをビルドすると .dylib が生成されるように、Android 向けに動的ライブラリをビルドすると .so になるだけで、.so が特別な何かというわけではなく、動的ライブラリの一種というだけなのです。

生成した .so を Android アプリに組み込む

jniLibs と AGP

Swift Package のビルド成果物である .so を Android アプリで利用するには、Android 側のビルド工程へ渡す必要があります。
AGP（Android Gradle Plugin）は、Gradle 上で APK の生成など、

Android 固有のビルド処理を担うプラグインです。Android には jniLibs というネイティブライブラリを配置するディレクトリがあり、ここに .so を配置することで AGP によって APK へ組み込まれます。
全体の流れは次の通りです。

```
Swift Package
        ↓ Android 向けにビルド
.build/<triple>/debug/*.so
        ↓ Gradle タスクで jniLibs へコピー
build/generated/jniLibs/<ABI>/*.so
        ↓ AGP が APK へ組み込む
APK/lib/<ABI>/*.so
```

ABI（Application Binary Interface）は、arm64-v8a など、CPU アーキテクチャに応じたバイナリの種類を表します。上記の流れでは build/generated/jniLibs を jniLibs として Gradle に設定し、Gradle タスクで ABI ごとに .so を配置しています。

ここへ配置するのは、libHelloLib.so のようなアプリ固有の .so だけではありません。libswiftCore.so や libswiftAndroid.so などの Swift ランタイムに加え、libSwiftJava.so などの関連ライブラリも jniLibs へ配置します。

Kotlin / Java とネイティブコードをつなぐ
JNI

Android アプリの多くは Kotlin / Java で書かれています。一方、Swift SDK for Android によってビルドされた Swift コードは、Android 向けのネイティブライブラリである .so として存在します。
この Kotlin / Java とネイティブコードをつなぐ仕組みが、JNI（Java Native Interface）です。JNI を利用する場合、Java 側には次のような native メソッドを宣言します。

```
private static native String $greeting();
```

native が付いたメソッドは、Java 側に処理の本体を持ちません。実行時には、ロードされた .so 内にある対応する関数が JNI を通じて呼び出されます。

ただし、JNI 自体が Swift を直接理解するわけではありません。JNI からの呼び出しを受け取り、元の Swift 関数へ中継する Swift 側の JNI 実装も必要です。
この橋渡しコードを毎回手作業で用意するのは大変です。そこで、native メソッドを含んだ Java Wrapper と、Swift 側の JNI 実装を生成する JExtract が役立ちます。

Java Wrapper と Swift 側の JNI 実装を生成する
JExtract

JExtract は、Swift の API を読み取り、Java から利用するための Binding を生成する swift-java のツールです。
JExtract には複数の生成モードがあります。Android では、Kotlin / Java とネイティブコードを JNI で接続するため、JNI モードを使用します。JNI モードでは、Java Wrapper と Swift 側の JNI 実装が生成されます。

Java Wrapper

Java Wrapper は、Kotlin / Java から利用するためのクラスです。Android 側のコードは、この Wrapper を通常の Java クラスと同じように呼び出します。
内部には、.so をロードする処理や、JNI を通じて Swift 側の実装へつながる native メソッドが含まれます。

Swift 側の JNI 実装

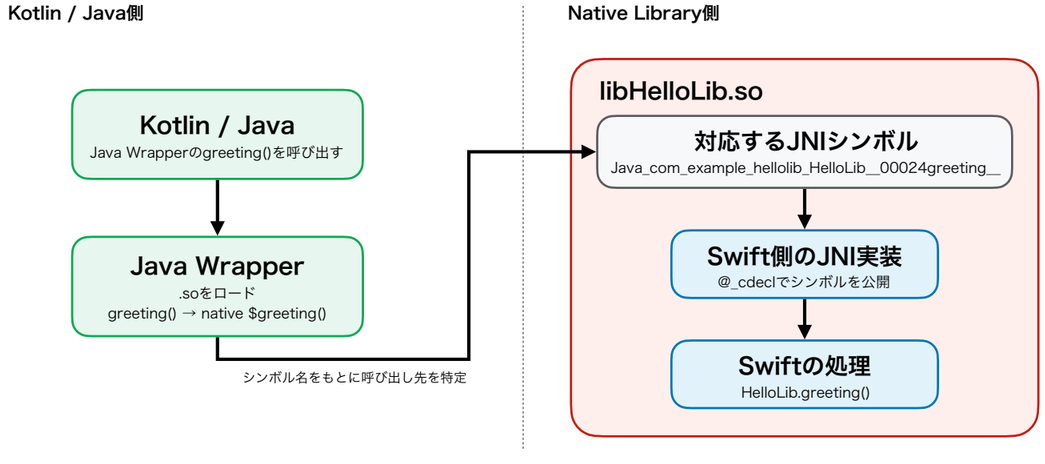
Swift 側の JNI 実装は、Java の native メソッドに対応する Swift 側の中継コードです。Java 側からこの中継コードを呼び出すには、.so 内の対応する関数を識別するための名前が必要です。この名前をシンボルと呼び、外部から関数を見つけるために使います。
JExtract が生成するコードでは、JNI の規則に沿ったシンボル名を @_cdecl に指定し、その名前で Swift 関数を公開します。

```
@_cdecl("Java_com_example_hellolib_HelloLib__00024greeting__")
```

これにより、Java の native メソッドから Swift 側の実装へ到達できます。

実行時の流れを見る

これまで例にしてきた HelloLib パッケージの greeting() が、アプリの実行時に Kotlin / Java からどのように呼び出されるのかを順番に追います。



1. Kotlin から Java Wrapper を呼ぶ

Android アプリ側では、生成された Java Wrapper を Kotlin から呼び出します。

```
import com.example.hellolib.HelloLib

val message = HelloLib.greeting()
```

2. .so をロードして native メソッドを呼ぶ

Java Wrapper では、最初に 2 つの共有ライブラリをロードします。

- libSwiftJava.so
  - swift-java が提供する Swift/Java 連携用のランタイム
  - Swift と Java の値変換など、JNI 連携に必要な機能を提供
- libHelloLib.so
  - greeting() と Swift 側の JNI 実装を含む
  - libSwiftJava.so に依存している

この依存関係を満たすため、Java Wrapper のクラス初期化時には、libSwiftJava.so、libHelloLib.so の順にロードします。このロードはクラス初期化時に一度だけ行われます。

```
public final class HelloLib {
    static final String LIB_NAME = "HelloLib";

    static {
        // libSwiftJava.so と libHelloLib.so をロードする
        SwiftLibraries.loadLibraryWithFallbacks(
            SwiftLibraries.LIB_NAME_SWIFT_JAVA
        );
        SwiftLibraries.loadLibraryWithFallbacks(
            LIB_NAME
        );
    }

    public static String greeting() {
        return $greeting();
    }
    // 実装は libHelloLib.so 内に存在
    private static native String $greeting();
}
```

コードを見ると、public な greeting() が private native メソッドの $greeting() を呼び出しています。$greeting() の実装は Java 側にはありません。呼び出し時には、JNI がシンボル名をもとに libHelloLib.so 内の対応する関数を特定します。

3. .so 内の Swift 側 JNI 実装に到達する

Java Wrapper の native メソッドを呼び出すと、JNI はシンボル名をもとに .so 内の対応する関数を特定し、Swift 側の JNI 実装を呼び出します。今回の $greeting() に対応するシンボルは、次の名前です。

```
Java_com_example_hellolib_HelloLib__00024greeting__
|    |                                |          |
|    |                                |          引数なし
|    |                                └ $greeting
|    └ パッケージ名とクラス名
└ JNI シンボルの接頭辞
```

JNI シンボルは、パッケージ名、クラス名、native メソッド名などを組み合わせて作られます。この例では、$greeting() の $ が Unicode の U+0024 を表す _00024 へ変換されています。そのため、Java 側の $greeting() と .so 内の長いシンボル名が対応します。

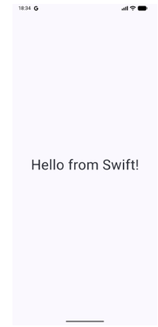
```
@_cdecl("Java_com_example_hellolib_HelloLib__00024greeting__")
public func Java_com_example_hellolib_HelloLib__00024greeting__(
    environment: UnsafeMutablePointer<JNIEnv?>!,
    thisClass: jclass
) -> jstring? {
    return HelloLib.greeting().getJNILocalRefValue(
      in: environment
    )
}
```

コードをよく見ると、HelloLib.greeting() を呼び出していることがわかります。

JNI をまたいで戻り値を返すには、Swift と Java の型を変換する必要があります。このコードでは、HelloLib.greeting() が返した Swift の String を、getJNILocalRefValue(in:) によって JNI の jstring へ変換しています。その値は native メソッドの戻り値として Java Wrapper へ返され、Kotlin / Java 側では String として受け取られます。

4. 元の Swift コードが実行される

最後に、greeting() が libHelloLib.so 内のネイティブコードとして実行され、"Hello from Swift!" が返ります。右の画面は、その文字列を Android アプリ上に表示したものです。
この例で Swift が担当しているのは文字列を返す処理までで、画面への反映は Android 側の UI が行っています。
Android アプリから見ると Kotlin / Java のメソッドを呼んでいるように見えますが、その先では Java Wrapper、JNI、.so 内の Swift 側 JNI 実装を経由して、元の Swift コードが実行されています。



おわりに

Swift SDK for Android の公式の Getting Started 記事やサンプルアプリを触り始めたところ、普段使わない技術が多く出てきて理解するのに時間がかかったため、今回の記事の執筆に至りました。
Skip で iOS / Android アプリをリリースしていますが、UI までも同一コードで管理しようとすると片方の OS では意図しない挙動が発生することも多くなり、マルチプラットフォームツールで UI のコードも統一するのは限界だと感じています。そこで Swift SDK for Android を用いた方法では、今のところビジネスロジックのみを共通化する思想であり、Swift 公式のツールであるため、今後の更なる発展を期待しています。

この記事を書いた人
yamaken



「Apple Store」で検索するとりんごの直売所がヒットする青森県で 1997 年に生まれる。大学に入ってから iOS アプリ開発に目覚め、個人アプリをリリースし始める。くるくる髪の天然物のパーマが特徴。Swift を Android 上で呼び出す Skip に興味をもち、try! Swift Tokyo 2025 では Skip についてのトークで登壇し、iOSDC Japan 2025 では Skip についての記事を執筆。Skip を採用した iOS / Android アプリ「MyFitsLog」をリリースして運用している。最近はさまざまなイベントで司会をやることが増えている。